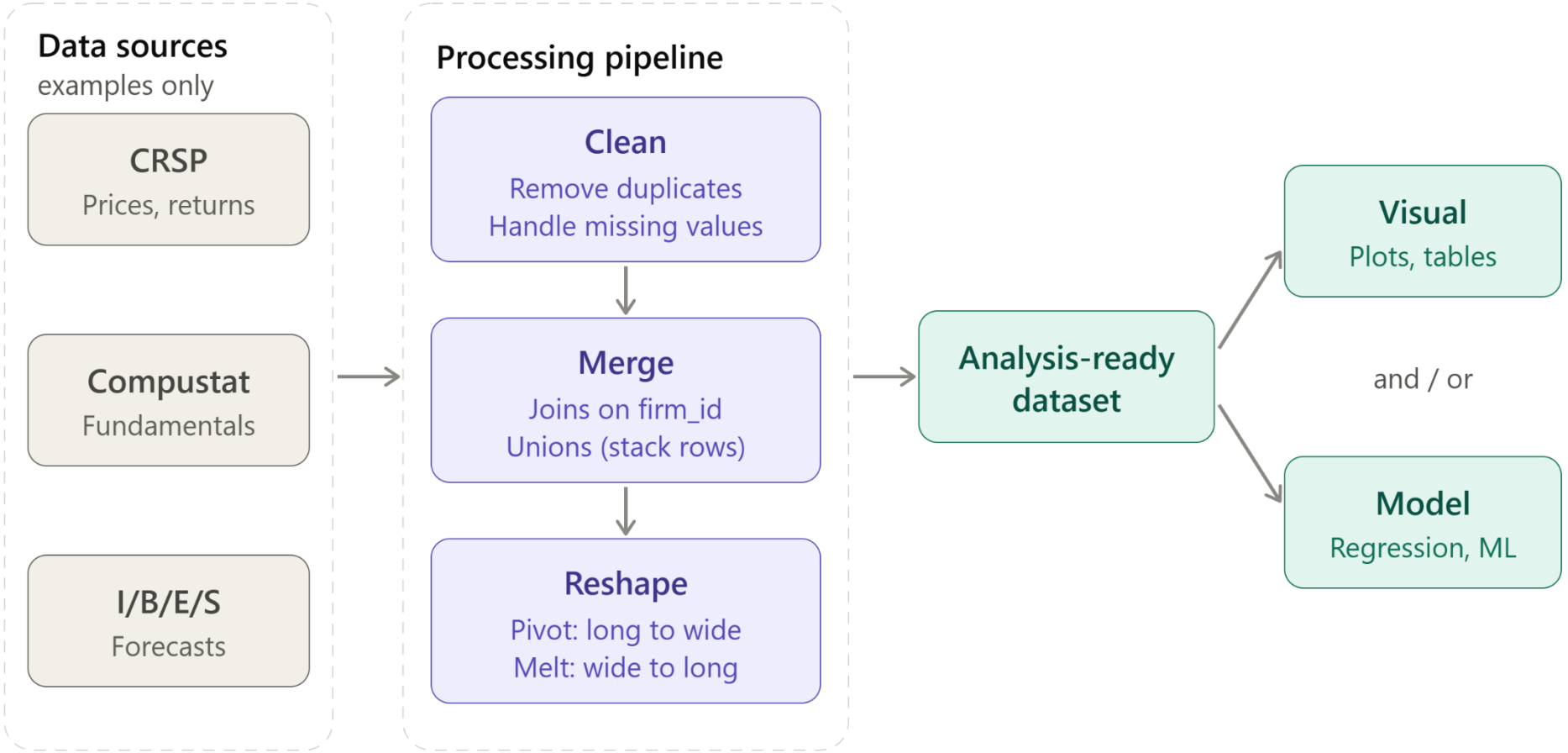
RICE | BUSINESS
Jones Graduate School of Business

Processing Data using Python

PhD Workshop 2026 Fall

Objectives - Empirical Research Pipeline

- Sourcing |Data processing |Analyses



Common Data Processing Techniques

- JOINS (Horizontal Mergers)
- UNIONS (Vertical Mergers)
- Group by (row aggregation)
- Transposition (Long-to-wide and wide-to-long)
- Calculated Fields
- Lags and Leads

Other Objectives

- Get familiar with the Visual Studio Code environment
- Design and implement a workflow structure
- Learn to inspect the data for weirdness (nulls, duplicates, etc.)
- Verify your own work

My environment – VS Code and Python

- VS Code
  - Extensions:
    - Python
    - Jupyter
    - Data Wrangler
- Python
  - Miniforge
    - Create a virtual environment called myenv1
- Step-by-Step Installation [link]

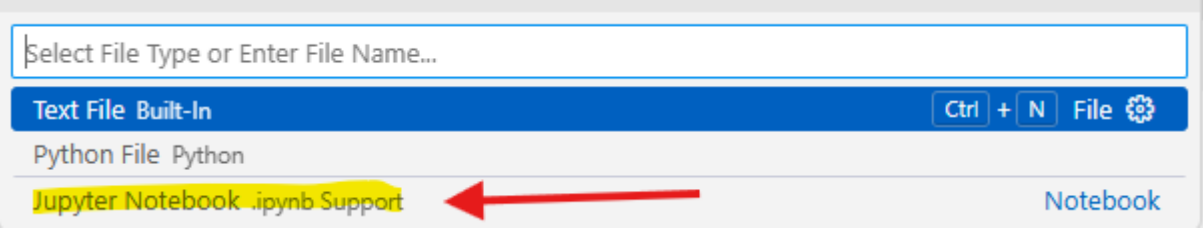
Manipulating Data in Python

- Dataframes
  - Very powerful and versatile
  - For large datasets, can use considerable RAM
  - Weak at ranged joins

- SQL
  - We can pull data from WRDS with SQL queries (PostgreSQL)
  - DuckDB
    - Uses PostgreSQL syntax
    - Can query CSVs, parquet selecting columns and filtering rows (less RAM)

The Visual Studio Code Environment

- Jupyter Notebook
  - My preferred method for exploring data and coding
  - Convenient way to create, organize, and execute code
  - File | New File | Jupyter Notebook



- Jupyter Notebook vs Python Script
  - Scripts useful to run processes in parallel (e.g., concurrent.futures)
    - Dr. Kerry Back's notes on notebooks and scripts (may be outdated)
  - Jupytext package run scripts blocks as cells and syncs notebooks/scripts

Example Workflow Structure

- How do I approach structuring empirical research pipeline tasks?
  - Step/Program1: Data Sources
    - Download all data to be used (e.g., WRDS, CSVs, APIs)
    - Save it into a folder to be read in the next step
  - Step/Program2: Read and merge data into a dataset for analysis and save
    - Example: If running a Fama-Macbeth Regression I need a dataset that has a column for the firm identifier, time period, dependent variable (e.g., Ret), independent variables (e.g., size, bm) in the appropriate time structure
      - Ret in t+1 and size and bm in t, or
      - Ret in t and size and bm in t-1
  - Step/Program3…N: One per analysis, ‘table’, and figure
    - Descriptive Statistics, Regression1, Regression2, Robustness1, Robustness2

How to Code?

- Develop your own conventions
  - Naming – variables, files, folder structures
  - Cell, functions, script/notebook use
- Keep in mind everything in Python uses RAM
  - Issues with large datasets (workaround: duckdb and parquet files)
- Recommendations
  - I like to have blocks that do one task, run it, and check the output
  - Long-running tasks: explore alternatives and pick fastest
  - Agentic AI: Make sure you ‘manually’ do everything you need to own

Joins – What are joins?

- Horizontal Merges
  - Inner Join
  - Left Join
  - Right Join
  - Full Join
  - Cross Join

Joins

- Join matches rows by the columns specified
- Types:
  - Inner = Requires rows to exist on both tables
  - Left/Right = Requires rows to exist on the specified side table and matches what it can from the other side
  - Full = Matches what it can, leaves empty rows for non matches
  - Cross join – Do you really need this? Matches all possible combinations of both tables
- https://www.codeproject.com/Articles/33052/Visual-Representation-of-SQL-Joins

Ex1 Joins

- Datasets
  - Forecasted EPS (df_frct)
  - Actual EPS(df_act)

- Data Dictionary
  - firm_id – unique firm identifier
  - eps_forecast – forecasted EPS, stock split adjusted
  - sps_actual – actual EPS, stock split adjusted

Ex1 Joins - Inner

- Example:

df_fcst

	firm_id	eps_forecast
0	1	2.10
1	2	1.85
2	3	3.40
3	4	0.95

df_act

	firm_id	eps_actual
0	2	1.90
1	3	3.40
2	4	1.10
3	5	2.60

```
>>> df_a_inner = pd.merge(df_fcst, df_act, how='inner', left_on=['firm_id'], right_on=['firm_id'],
suffixes=('','_x'))
```

df_a_inner: 3 rows × 3 columns

	firm_id	eps_forecast	eps_actual
0	2	1.85	1.90
1	3	3.40	3.40
2	4	0.95	1.10

firm_id in both tables unmatched firm_id 1 and 5 are dropped

Ex1 Joins - Left

- Example:

df_fcst

	firm_id	eps_forecast
0	1	2.10
1	2	1.85
2	3	3.40
3	4	0.95

df_act

	firm_id	eps_actual
0	2	1.90
1	3	3.40
2	4	1.10
3	5	2.60

```
>>> df_b_left = pd.merge(df_fcst, df_act, how='left', left_on=['firm_id'], right_on=['firm_id'],
suffixes=('','_x'))
```

df_b_left: 4 rows × 3 columns

	firm_id	eps_forecast	eps_actual
0	1	2.10	NaN
1	2	1.85	1.90
2	3	3.40	3.40
3	4	0.95	1.10

firm_id in both tables forecast only

Ex1 Joins - Right

- Example:

df_fcst

	firm_id	eps_forecast
0	1	2.10
1	2	1.85
2	3	3.40
3	4	0.95

df_act

	firm_id	eps_actual
0	2	1.90
1	3	3.40
2	4	1.10
3	5	2.60

```
>>> df_c_right = pd.merge(df_fcst, df_act, how='right', left_on=['firm_id'], right_on=['firm_id'],
suffixes=('','_x'))
```

df_c_right: 4 rows × 3 columns

	firm_id	eps_forecast	eps_actual
0	2	1.85	1.90
1	3	3.40	3.40
2	4	0.95	1.10
3	5	NaN	2.60

firm_id in both tables actual only

Ex1 Joins - Outer

- Example:

df_fcst

	firm_id	eps_forecast
0	1	2.10
1	2	1.85
2	3	3.40
3	4	0.95

df_act

	firm_id	eps_actual
0	2	1.90
1	3	3.40
2	4	1.10
3	5	2.60

```
>>> df_d_outer = pd.merge(df_fcst, df_act, how='outer', left_on=['firm_id'], right_on=['firm_id'],
suffixes=('','_x'))
```

df_d_outer: 5 rows × 3 columns

	firm_id	eps_forecast	eps_actual
0	1	2.10	NaN
1	2	1.85	1.90
2	3	3.40	3.40
3	4	0.95	1.10
4	5	NaN	2.60

firm_id in both tables forecast only actual only

Ex1 Joins - Cross

- Example:

df_fcst

	firm_id	eps_forecast
0	1	2.10
1	2	1.85
2	3	3.40
3	4	0.95

df_act

	firm_id	eps_actual
0	2	1.90
1	3	3.40
2	4	1.10
3	5	2.60

```
>>> df_e_cross = pd.merge(df_fcst, df_act, how='cross', suffixes=('','_x'))
```

	firm_id	eps_forecast	firm_id_x	eps_actual
0	1	2.10	2	1.90
1	1	2.10	3	3.40
2	1	2.10	4	1.10
3	1	2.10	5	2.60
4	2	1.85	2	1.90
5	2	1.85	3	3.40
6	2	1.85	4	1.10
7	2	1.85	5	2.60

	firm_id	eps_forecast	firm_id_x	eps_actual
8	3	3.40	2	1.90
9	3	3.40	3	3.40
10	3	3.40	4	1.10
11	3	3.40	5	2.60
12	4	0.95	2	1.90
13	4	0.95	3	3.40
14	4	0.95	4	1.10
15	4	0.95	5	2.60

Ex1 - Calculations

- Hands on practice
  - Calculate the EPS surprise

	firm_id	eps_forecast	eps_actual	surp	surp_fill0
0	1	2.10	NaN	NaN	-2.10
1	2	1.85	1.90	0.05	0.05
2	3	3.40	3.40	0.00	0.00
3	4	0.95	1.10	0.15	0.15
4	5	NaN	2.60	NaN	2.60

firm_id in both tables forecast only actual only

Unions in Pandas

- Vertical Joins (called concatenation in Pandas)
  - Create a table with forecasts and actuals in long format
  - Create a list of unique IDs from both tables

	firm_id	type	eps
0	1	forecast	2.10
1	2	forecast	1.85
2	2	actual	1.90
3	3	forecast	3.40
4	3	actual	3.40
5	4	forecast	0.95
6	4	actual	1.10
7	5	actual	2.60

from df_fcst from df_act

	firm_id
0	1
1	2
2	3
3	4
4	5

Python – Try it out

- Open Ex1_joins_and_unions
- (My) Version of Pyt...
  - myenv1 (Python 3.14.8)
  - You can have more than one interpreter; there are ways to set the default one in VS Code
  - Menu:



  - Jupyter Variables

Jupyter Variables

Name	Type	Size	Value
df_a_inner	DataFrame	(3, 3)	firm_id eps_forecast eps_actual
df_act	DataFrame	(4, 2)	firm_id eps_actual
df_b_left	DataFrame	(4, 3)	firm_id eps_forecast eps_actual
df_c_right	DataFrame	(4, 3)	firm_id eps_forecast eps_actual
df_d_outer	DataFrame	(5, 3)	firm_id eps_forecast eps_actual
df_e_cross	DataFrame	(16, 4)	firm_id eps_forecast firm_id_x eps_actual
df_fcst	DataFrame	(4, 2)	firm_id eps_forecast

  - Data Wrangler features:
    - Filter and sorting data with GUI

Questions?

Introduction to SQL

Select = Specify columns and in what order

From = source dataset (or sources when merging)

Where = filter data

Group by = apply aggregated functions

Having = filter data based on aggregated functions

Order by = Sort dataset

- WRDS and DuckDB use PostgreSQL syntax
- Example:
  - SELECT date, rf, mktrf, smb, hml, umd
    FROM ff_all.factors_monthly
    ORDER BY date

- Note: Common Table Expressions (CTE) can also be handy to perform multiple operations

How to get data from WRDS

- Identify the tables and column names you need
- Install the wrds package if you don't have it
  - !pip install --no-deps wrds
    - In Jupyter, the ! runs a shell command from a cell
    - --no-deps stops wrds from forcing older pinned versions of pandas and numpy
  - Make sure its dependencies are installed:pandas, psycopg[binary], sqlalchemy
  - !pip install pandas "psycopg[binary]" sqlalchemy
- Restart the kernel after installing 

  - Do not restart the kernel if you have an open connection
- Write the SQL query as a string and load the result into a dataframe
  - Optional: specify any date columns
- Execute

Storing your work

- Save your program
  - If you opened an already created notebook, you can simply save to overwrite your changes
  - If it is a new file, you choose a name and path to save it, you don’t need to worry about the file extension (it will usually say JSON)

- Save your data
  - I like to split my programs by step in the workflow process
    - Download
    - Merge and cleanup
  - I save the downloaded data frames as parquet files

Why SQL matters?

- Using SQL in Python
  - For WRDS downloads be familiar with select, from, and where statements
  - DuckDB let's you use SQL on data frames and other data types (e.g., csv)
  - You can leverage Common Table Expressions to do multiple operations

- Note: For both WRDS and DuckDB use PostgreSQL syntax

Ex2 – Download and Process

- In one program, generate code that will get ff3 and ff5 tables
  - Create a SQL query
  - Identify the tables and columns names as needed
  - Specify date columns
    - Date columns in SQL do not automatically are identified as datetimes in a dataframe
  - Save dataframes to parquet files

- In another program, open the parquet files, compute descriptive statistics on smb from ff3 and ff5 and run correlation

Ex2_99 helper programs

99 marks the extras outside the numbered steps. Each comes as a .py file or a notebook, so use either. Not fully tested.

Ex2_99_wrds_libraries

FIND NAMES

Lists every WRDS library (schema) in your subscription with all its tables as schema.table, so you know the exact names to use.

Ex2_99_download_table

EXPLORE ONE TABLE

Easily download one table to a dataframe to explore. Pick the schema and table, limit the rows if you like, and see each column with its type and comment.

Ex2_99_download_tables_stream

DOWNLOAD MANY

Downloads a list of tables straight to parquet in chunks, so memory is never the limit. Files go in one folder per schema, compressed with zstd.

Ex2_99_terminate_connections

CLEAN UP

Frees WRDS connections left open by earlier sessions. It lists your connections and ends the idle ones. A dry run is on by default.

Aggregate Functions

- Collapse rows into functions to calculate:
  - Sum
  - Count
  - Avg
  - Std
  - Median

- Depending on your tool this is achieved in different ways, but the concept remains the same

Basic Aggregation

- Dimensions and Measures
  - For analysis we usually want dimensions as rows and measures as columns (Tableau Terminology)

- Let’s look at the data (open Ex3-GroupBy Example.xlsx in Excel):
  - 6 rows
  - 2 dimensions (Store and Salesperson), 1 measure (Amount)

Aggregate data

- Aggregate data by:
  1. Store
  2. Salesperson
  3. Store and Salesperson

Ex3 – Aggregation and Rank

- Aggregate sales by 2-digit SIC code per year and rank them
  - Open Ex3_01_download
  - Change wrds username and download path
  - run
- Algorithm concerns
  - We need to make certain choices:
    - How do we calculate the year? [Differences between fyear and datadate]
    - How do we extract the first 2-digits of each SIC code? (trivial with AI)
- Economic interpretation concerns
  - Do you see any issues when interpreting sales per industry per year?

Look at the data (always look at the datasets)

- SIC = Standard Industry Classification
  - https://www.sec.gov/corpfin/division-of-corporation-finance-standard-industrial-classification-sic-code-list
- We have 2 SIC codes columns (SIC and SICH)
  - Compustat has a backfilled sic code (SIC) and a historical one (SICH)
    - Backfilled means that the most recent value of the SIC code was applied to the entire time-series of the firm
    - Do you see any potential issues with this?
- For this exercise, let's use SIC, but be aware of what that entails

Ex3 Solution

- Create columns with 2-digit sic code and year
  - Floor(sic/100) as industry
  - Use fyear for yearly variable

- Aggregate by industry, fyear

- Let’s get top 10 industries by year

- Discuss sich and sic differences (Ex3_03_sich_changes)

Questions?

What is a duplicate

- A duplicate is a record that is multiple times in your dataset
  - Exact Duplicate
  - Partial Duplicate
  - Interpretation Duplicate

- Why would a duplicate be an issue?

Exact Duplicate

- An exact duplicate is two or more rows that have the exact values in all fields
  - Example (customer master table)

CustomerID	FirstName	LastName	Phone	Email
1	Kerry	Swift	(281)111-1111	kerry.swift@a.com
1	Kerry	Swift	(281)111-1111	kerry.swift@a.com

Partial Duplicate

- A partial duplicate is two or more rows that have the exact values in some fields but not in others and refer to the same entity
  - Example (customer master table)

CustomerID	FirstName	LastName	Phone	Email
1	Kerry	Swift	(281)111-1111	kerry.swift@a.com
1	Kerry	Swift	(713)111-1111	kerry.swift@a.com

Interpretation Duplicate

- An interpretation duplicate is two or more rows that are unique but pose a duplicate problem because of how you are interpreting the data

CustomerID	FirstName	LastName	Phone	Email
1	Kerry	Swift	(281)111-1111	kerry.swift@a.com
2	Alex	Smith	(713)111-1111	kerry.swift@a.com

- Another example…

Another Interpretation Duplicate example

- You are constructing a dataset where you need accounting firm financials for every year. Your structure requires you to have one row per firm-year.

gvkey	tic	conm	datadate	Assets	Liabilities	Sales	COGS	XSGA	y
7116	MAT	MATTEL INC	19840131	611.039	744.34	633.413	318.284	227.367	1984
7116	MAT	MATTEL INC	19841231	512.693	381.341	880.85	430.034	312.233	1984

- It isn't clear how to best handle this, depends on many factors.
- We have other issues when merging (e.g. 2 PERMNOs to 1 GVKEY)

Ex4 - How to check for duplicates?

- Let’s use the compustat dataset from Ex3 and find firms that have the same fyear

- Create a column that extracts the year from datadate, call it y and check duplicates where a gvkey has more than one y

Ex4-Managing Duplicates

- For gvkeys that have more than 1 y choose:
  - The one with the highest assets
  - The one with the most recent datadate

Lags and Leads

- We want lags when we are calculations growth ratios
- See Excel example lag prop
- We can use shift in Python
  - Make sure your dataset does not contain duplicates
  - Careful when time series data has gaps
  - Shift takes the previous row value indiscriminately, so prep your dataframe accordingly if you do down this route

- Examples: Asset Growth and Sales Growth

Ex5 – Lags and Leads

- Using the Compustat dataframe calculate Asset Growth and Sales Growth

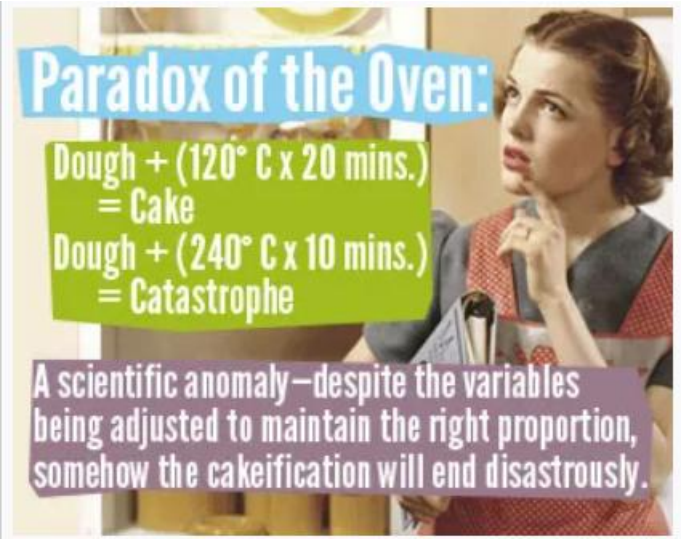
- Discuss
  - Missing Values
  - 0s

Ex6-Ranged Joins

- We have a linking table that has start and end dates for links between identifiers of 2 databases
- Pandas doesn’t have a way to apply a range as part of the join condition
- We can either:
  - Join on Identifier which will create multiple rows per original row, then filter that the date falls between the start and end date
    - Can blow up RAM in the first step if many links
  - Use DuckDB!
- Do we have duplicates?

Data Engineering - Code Optimization

- Tradeoffs:
  - CPU usage (we are not baking)
  - RAM vs Disk
  - I/O
    - Are you doing many small reads, or a few big ones?
    - Are you disposing of workers and creating new ones too many times?
  - Coding time investment vs fastest running code (what is fast enough?)
    - How many times will you have to re-run it?



Ex8-Replicate Beta and Idiosyncratic Volatility

- Paper
  - Shue, Kelly, and Richard Townsend, 2021, Can the market multiply and divide? Non-proportional thinking in financial markets. *Journal of Finance*, 76(5), 2307–2357.

- Their methodology is based on Lewellen and Nagel (2006)
- We'll join, lag, filter, and calculate new measures
- We'll also run regressions
- How to replicate papers

Ex8-Plan

- Basic regression for CAPM

The basic framework for our tests is standard. We focus on time-series CAPM regressions for a handful of stock portfolios:

$$R_{it} = \alpha_i + \beta_i R_{Mt} + \varepsilon_{it}, \quad (5)$$

where R_{it} is the excess return on portfolio i and R_{Mt} is the excess return on the market. The

- Lewellen and Nagel (2006) adjustment for non-synchronous price movements

Also, following Dimson (1979), we include both current and lagged market returns in the regressions, estimating beta as the sum of the slopes on all lags (alpha is still just the intercept). For daily returns, we include four lags of market returns, imposing the constraint that lags 2–4 have the same slope to reduce the number of parameters:

$$R_{i,t} = \alpha_i + \beta_{i0} R_{M,t} + \beta_{i1} R_{M,t-1} + \beta_{i2}[(R_{M,t-2} + R_{M,t-3} + R_{M,t-4})/3] + \varepsilon_{i,t}. \quad (7)$$

The daily beta is then $\beta_i = \beta_{i0} + \beta_{i1} + \beta_{i2}$. (Adding a few more lags doesn't affect the results.)

- Shue and Townsend basic filters

To classify stocks by nominal share price, past returns, etc., we always use lagged information to avoid look-ahead bias. To reduce the influence of outliers, we restrict our baseline analysis to stock-months in which (1) there are at least 15 trading days with non-missing return data in CRSP and (2) there is variation in the stock's price (i.e., the stock price is not the same on all trading days within the month), and (3) the nominal stock price at the end of the previous month was not in the top or bottom 1% of stock prices that month.

Ex8-Plan (Cont.)

- ‘Ingredients”
  - Dataset with daily firm returns
  - Dataset with daily market excess returns and risk free rate
  - Dataset with closing month lagged prices (t-1)
- Steps:
  - Create lags for market returns (t-1, t-2, t-3, t-4)
  - Calculate the average returns for t-2 to t-4
  - Join the daily market returns by date to the firm returns
  - Calculate firm excess returns (ret-rf) for every day

Ex8-Plan (Cont.)

- Dataset should have columns :permno,year-month,retrf,mktrf,mktrftm1,mktrfavgtm2_4
- Run a regression per firm ym:
  - Retrf = mktrf,mktrftm1, mktrfavgtm2_4
  - Collect the 3 beta coefficients and add(can be done later)
  - Calculate standard deviation of the residuals for ivol and annualize $\sqrt[2]{252}$
- For every year-month:
  - Calculate percentile rank (cross-sectionally) of the lagged closing price and filter out 1 and 99 (rank them in 100 groups and drop first and last)

Ex8-Plan (Cont.)

- Running this many regressions is computationally intensive and also uses a lot of memory because it is daily data. Many considerations come at play (~3.8M regressions):
  - Parallelized with concurrent futures:
    - 1 hour 20 minutes ✓ 81m 35.9s
  - My previous attempt on this problem with concurrent futures
    - 3 hours 57 minutes ✓ 237m 34.9s

Ex8-Plan (Cont.)

- Get a final dataset with permno, year-month, ivol, beta, absbeta
- Calculate descriptive statistics and compare (their sample ends in 2016 but we can extend it and see):

Panel A: Summary

	Obs	Mean	Median	Std Dev
Total Volatility (Annualized)	3,254,302	0.510	0.390	0.450
Idiosyncratic Volatility (Annualized)	3,254,302	0.448	0.334	0.415
Market Beta	3,254,302	0.941	0.809	3.069
Price	3,254,302	18.85	13.50	19.13
Market Capitalization (Millions)	3,254,302	1179.9	64.47	8800.6
Insitutional Ownership	2,165,251	0.346	0.272	0.292
Sales Volatility	2,316,178	0.273	0.0966	0.691
Volume Turnover	2,996,292	0.0882	0.0382	0.204
Market-to-Book Ratio	2,240,583	1.977	1.252	18.19
Book Leverage	2,130,604	0.232	0.191	0.278
Bid Ask Spread (Percentage)	2,246,545	4.840	2.525	17.63
Intraday Price Range (Percentage)	2,555,175	3.586	2.826	2.726

Appendix: Miniforge vs Anaconda (or Miniconda)

- Potential licensing issues
- Anaconda now has Academic Licensing

- We swapped Anaconda for Miniforge in JGSRC1
- Other Universities have migrated to Miniforge as well
  - MSU


RICE | BUSINESS
Jones Graduate School of Business